

Rapport de Projet - Chatbot Portfolio IA

Introduction

Ce projet consiste à créer un chatbot RAG (Retrieval-Augmented Generation) pour répondre aux questions sur mon parcours. Il combine une base vectorielle, un agent IA avec outils, et une interface utilisateur Streamlit.

Architecture Générale

Le projet repose sur trois composants principaux :

1. **ingest.py** : Indexation des données dans la base vectorielle
2. **agent.py** : Logique de l'agent IA
3. **app.py** : Interface utilisateur Streamlit

1. Indexation des Données (**ingest.py**)

Étapes principales

Préparation des données

- Fichiers Markdown dans **data/**, structurés par titres (**#**, **##**).

Chunking intelligent

```
def improved_chunking(content, max_char=1000):
```

- Découpage par sections (H2), ajout de contexte global (H1).
- Redécoupage si une section dépasse 1000 caractères.

Indexation dans Upstash Vector

```
index.upsert(vectors=all_vectors)
```

- Métadonnées enrichies : **source**, **text**, **section**.
- Envoi groupé des vecteurs pour optimiser les performances.

2. Agent IA (**agent.py**)

Fonctionnement

Cycle principal

1. Préparation des messages.
2. Appel à l'API OpenAI.
3. Utilisation des outils si nécessaire.
4. Génération de la réponse.

Outils disponibles

1. `search_portfolio()` : Recherche sémantique dans la base vectorielle.
2. `get_current_time()` : Réponse sur la date/heure.

Personnalité de l'agent

```
instructions = (  
    "Tu es le jumeau virtuel de Rémi, étudiant en Science des Données. "  
    "Tu parles TOUJOURS à la première personne."  
)
```

Système de mémoire avec Redis

- Sauvegarde des conversations en JSON.
- Sessions triées chronologiquement avec Redis.

3. Interface Utilisateur (`app.py`)

Structure

Configuration

```
st.set_page_config(  
    page_title="Chat avec Rémi",  
    page_icon="🤖",  
    layout="wide"  
)
```

Sidebar interactive

- Gestion des sessions.
- Statistiques en temps réel.
- Liens externes.

Zone de conversation

```
for message in st.session_state.messages:  
    with st.chat_message(role, avatar=avatar):
```

```
st.markdown(content)
```

- Affichage de l'historique.
- Spinner pendant la génération.

Gestion de l'état

```
if "session_id" not in st.session_state:  
    st.session_state.session_id = str(uuid.uuid4())
```

4. Déploiement

Configuration

- **Fichiers requis** : `requirements.txt`, `.env.example`.
- **Secrets nécessaires** :

```
OPENAI_API_KEY=...  
UPSTASH_VECTOR_REST_URL=...  
UPSTASH_REDIS_REST_URL=...
```

- **Clé API personnalisée** : J'ai créé ma propre clé API pour ChatGPT afin d'utiliser ce projet dans un contexte réel.
- **URL de l'application** : [Chatbot Portfolio](#)
- **Commande** : `streamlit run app.py`.

Points Techniques

1. **Gestion d'erreurs robuste** : Blocs `try/except` pour les appels externes.
2. **Optimisation** : Envoi groupé des vecteurs, `top_k=5` pour limiter les résultats.
3. **Expérience utilisateur** : Interface fluide, avatar personnalisé, statistiques.
4. **Modularité** : Séparation claire des responsabilités.

Conclusion

Le projet répond aux consignes du README :

- ✓ **Étape 1** : Données Markdown structurées
- ✓ **Étape 2** : Chunking intelligent
- ✓ **Étape 3** : Indexation dans Upstash Vector
- ✓ **Étape 4** : Agent IA avec OpenAI

- ✅ **Étape 5** : Outil RAG `search_portfolio`
- ✅ **Étape 6** : Interface Streamlit intuitive
- ✅ **Étape 7** : Déploiement sur Streamlit Cloud
- ✅ **Bonus** : Mémoire des conversations avec Redis

L'architecture est robuste, optimisée et offre une expérience utilisateur fluide.